

Отчёт - Чекпоинт 3: Интерфейс и развертывание

1. Описание функционала интерфейса

Реализован веб-интерфейс на базе Streamlit, обеспечивающий интуитивное взаимодействие пользователя с системой RAG. Интерфейс разделен на три основные области: боковую панель настроек, основную область ввода запросов и область отображения результатов.

1.1 Боковая панель настроек

Боковая панель предоставляет доступ к настройкам работы системы:

- **Режим работы:** Выбор одного из трех режимов поиска:
 - **Локальный RAG** — поиск по локальному корпусу статей arXiv
 - **Web-RAG** — поиск по интернету через Tavily API
 - **Гибридный локальный (DAT)** — комбинированный поиск с динамической настройкой весов
- **Топ-к документов:** Слайдер для настройки начального количества извлекаемых документов (диапазон: 3-10, значение по умолчанию: 5). Планировщик запросов может автоматически увеличить это значение на 3 при недостаточном качестве ответа

1.2 Основная область интерфейса

Центральная часть интерфейса содержит:

- **Заголовок и описание:** Логотип микроскопа, название “Science RAG” и краткое описание функциональности системы
- **Поле ввода вопроса:** Текстовое поле для ввода научного вопроса с поддержкой многострочного ввода
- **Кнопка “Получить ответ”:** Кнопка для запуска процесса поиска и генерации ответа

1.3 Область ответа

После выполнения запроса система отображает:

- **Процесс уточнения запроса:** При срабатывании планировщика автоматически показывается информация об итерациях уточнения, примененных стратегиях улучшения (переформулировка запроса, увеличение top-k на 3, или обе стратегии одновременно) и история всех запросов в раскрывающейся секции
- **Ответ:** Текст ответа, сгенерированный моделью Mistral на основе найденных документов
- **Метрики качества:** Средний и максимальный score релевантности найденных документов

1.4 Раздел источников

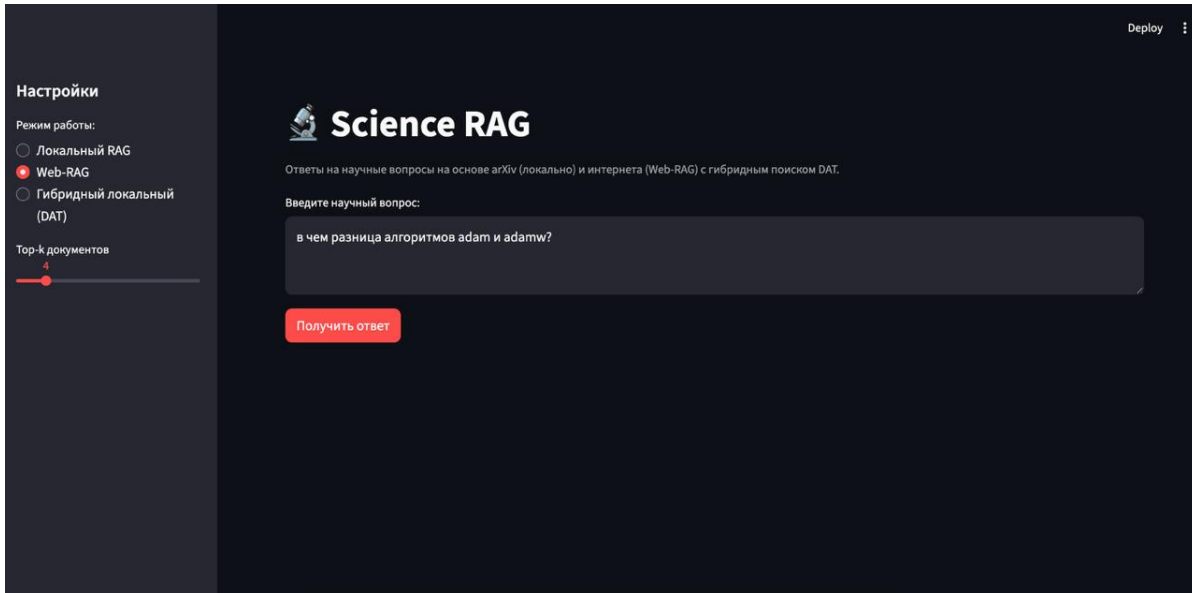
В нижней части интерфейса отображаются использованные источники:

- **Список источников:** Каждый источник представлен в раскрывающемся блоке с информацией:
 - ID документа и тип источника (local/web)
 - Score релевантности (от 0 до 1)
 - Заголовок документа (при наличии)
 - Ссылка на оригинальный источник (для web-источников)
 - Первые 1500 символов текста документа
- **Детали источника:** При раскрытии блока доступна полная информация о документе, включая математические формулы и подробные объяснения

2. Скриншоты интерфейса

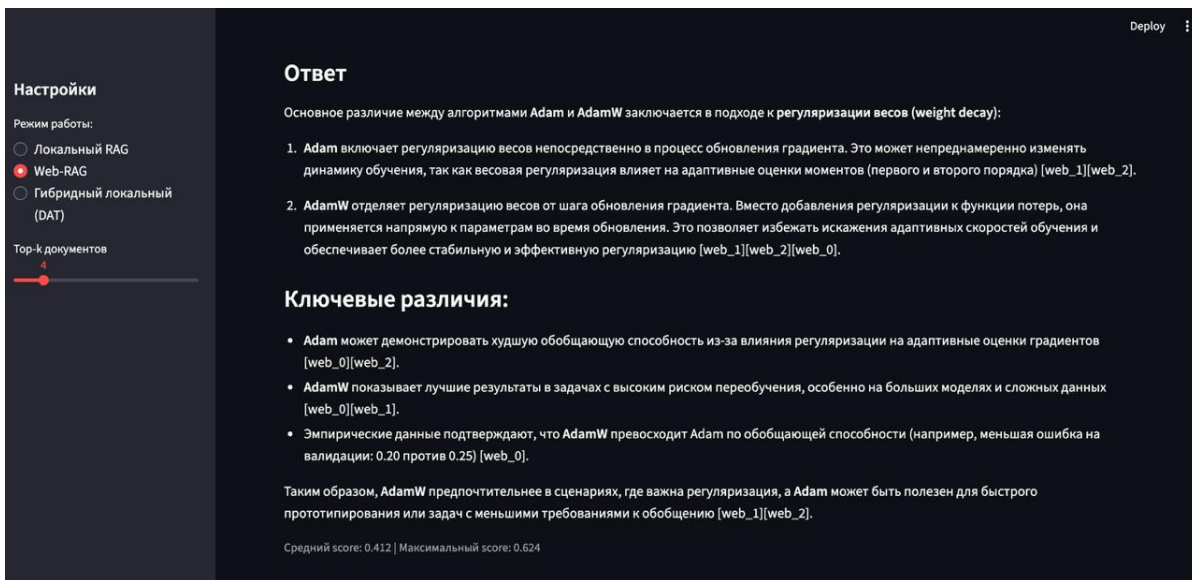
Скриншот 1: Главный интерфейс с настройками

На скриншоте представлена структура интерфейса: боковая панель с настройками режима работы (выбран режим “Web-RAG”) и параметра top-k документов (значение 4), основная область с полем ввода вопроса и примером запроса “в чем разница алгоритмов adam и adamw?”.



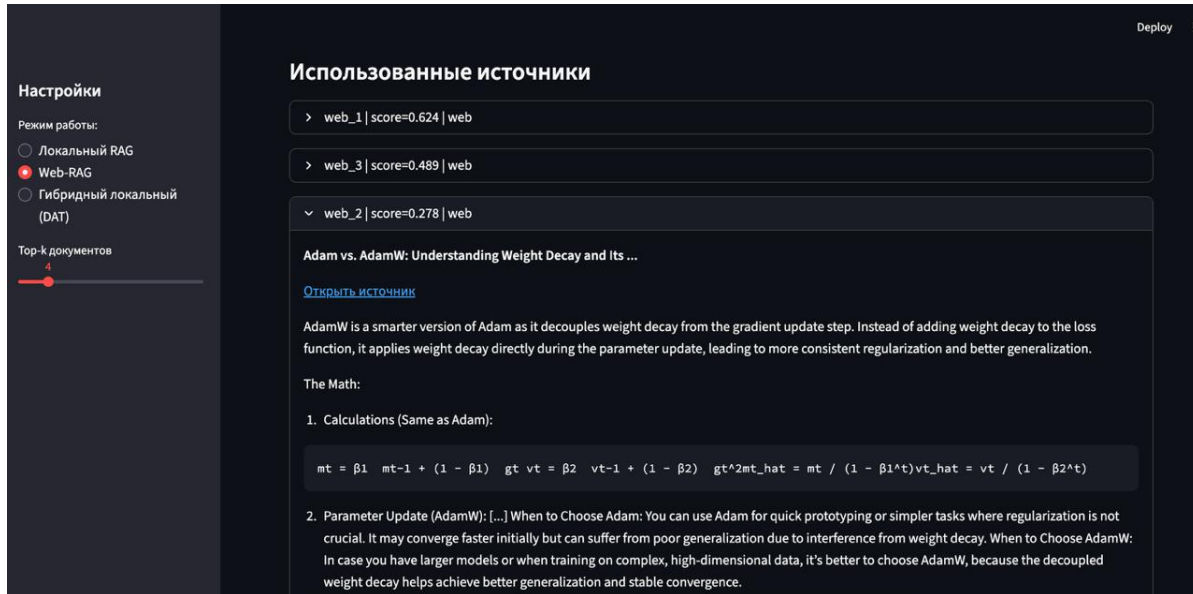
Скриншот 2: Ответ с метриками качества

На скриншоте показан результат работы системы: подробный ответ на русском языке с объяснением различий между алгоритмами Adam и AdamW, ссылками на источники [web_0], [web_1], [web_2], и метриками качества (средний score: 0.412, максимальный score: 0.624).



Скриншот 3: Развернутые источники

На скриншоте показана детальная информация об источниках: при раскрытии блока отображается полное содержание документа web_2 с заголовком статьи “Adam vs. AdamW: Understanding Weight Decay and Its...”, ссылкой на оригинальный источник, описанием, математическими формулами и рекомендациями по выбору алгоритма.



3. Инструкция по запуску

3.1 Запуск Streamlit-приложения

Для запуска приложения необходимо установить переменные окружения с API ключами:

```
export MISTRAL_API_KEY="..."
export TAVILY_API_KEY="..."
```

```
streamlit run app/streamlit_app.py
```

После запуска приложение будет доступно по адресу <http://localhost:8501>. В интерфейсе доступны три режима работы:

- **Local RAG** — поиск по локальному корпусу arXiv
- **Hybrid RAG (DAT)** — гибридный поиск с динамической настройкой весов
- **Web-RAG** — поиск по интернету через Tavily API

3.2 Запуск с Docker

Система поддерживает развертывание через Docker Compose для упрощения установки и запуска.

Требования

- Установленные Docker и Docker Compose
- API ключи для Mistral и Tavily

Подготовка

1. Создайте файл `.env` в корне проекта со следующим содержимым:

```
MISTRAL_API_KEY=your_mistral_api_key_here
TAVILY_API_KEY=your_tavily_api_key_here
```

2. Убедитесь, что данные подготовлены: индексы FAISS и BM25 должны быть созданы (см. раздел 4 в отчете чекпоинта 1).

Запуск Для сборки и запуска контейнера выполните:

```
docker-compose up --build
```

После успешного запуска приложение будет доступно по адресу `http://localhost:8501`

4. Финальные метрики качества системы

Результаты тестирования всех трёх retrievers на валидационной выборке из 859 вопросов:

Retriever	R@1	R@3	R@5
BM25	0.356	0.471	0.508
Dense	0.161	0.263	0.312
Hybrid + DAT	0.232	0.346	0.455

Анализ результатов

BM25 демонстрирует наилучшие результаты по всем метрикам: - Recall@1 = 0.356: в 35.6% случаев релевантный документ находится на первой позиции - Recall@5 = 0.508: более половины релевантных документов присутствуют в топ-5 результатах - Эффективность BM25 объясняется точным совпадением ключевых слов на терминологических запросах

Dense retrieval показывает более низкие результаты: - Recall@1 = 0.161: семантический поиск менее точен для точного попадания на первую позицию - Recall@5 = 0.312: семантический поиск находит релевантные документы, но с меньшей точностью позиционирования

Hybrid + DAT занимает промежуточное положение: - Recall@1 = 0.232: превосходит Dense, но уступает BM25 - Recall@5 = 0.455: близок к результатам BM25, что подтверждает эффективность комбинирования методов - Динамическая настройка весов α позволяет адаптировать поиск под каждый запрос, однако требует дальнейшей оптимизации

Выводы: - BM25 остается сильным baseline для данного типа запросов - Hybrid+DAT демонстрирует потенциал для улучшения через более точную настройку весов (например, LLM-based scoring α) - Система готова к использованию в production с учетом ограничений каждого метода поиска